

**Universidad Autónoma de Chiapas,
Facultad de Negocios Campus IV, Tapachula, Chiapas.**

Título:

Mini Intérprete de Operaciones Aritméticas y Manejo de Cadenas en NASM + C

Alumnos:

Diego Daniel Palafox Sánchez

Axel Yeray Cruz Arreola

Nombre del docente:

René Servando Rivera Roblero

Materia:

Traductores de Bajo Nivel

Semestre y grupo:

5to "G"



1.Introduccion

El presente proyecto tiene como objetivo integrar dos lenguajes fundamentales en el desarrollo de software de bajo nivel: **C** y **NASM (Netwide Assembler)**.

La finalidad es desarrollar un programa modular donde el módulo principal en C gestione la interacción con el usuario, mientras que las operaciones aritméticas y de cadenas se implementan en NASM, ejecutándose de manera eficiente directamente sobre la arquitectura x86 en modo de 32 bits.

En nuestro proyecto aplicamos los puntos de:

- Manejo de pila
- Convenciones de llamada entre C y ensamblador
- Procesamiento de cadenas
- Uso de macros en NASM
- Estructuras de control y ciclos
- Integración de módulos en un ejecutable final

2. Fundamentación Teórica

La interacción entre C y NASM requiere comprender el funcionamiento de las convenciones de llamada, el manejo de registros y la estructura de la pila dentro de la arquitectura x86. En este proyecto se emplea la convención **cdecl**, la cual establece que los parámetros se colocan en la pila y el valor de retorno se almacena en el registro EAX. Cada rutina escrita en NASM utiliza las instrucciones `push ebp`, `mov ebp, esp` al iniciar y `mov esp, ebp` seguido de `pop ebp` al finalizar, lo que garantiza un marco de pila estable para manipular parámetros y variables internas. Las rutinas aritméticas emplean registros como EAX, EBX y ECX para realizar operaciones directas sobre valores enteros mientras que las rutinas de cadenas recorren la memoria carácter por carácter utilizando ciclos y comparaciones. La integración de macros simplifica la estructura de las funciones al encapsular tareas repetitivas como el guardado y restauración del stack frame, logrando un código ensamblador más limpio y consistente. Cada uno de estos elementos refleja el funcionamiento interno de los programas y fortalece la comprensión del flujo de ejecución en bajo nivel.

2.1 Lenguaje C

El lenguaje C es un lenguaje de programación de propósito general, ampliamente utilizado en sistemas operativos, controladores y software de bajo nivel. Es ideal para integrarse con ensamblador debido a su cercanía al hardware y su soporte directo para llamadas externas.

2.2 Lenguaje Ensamblador NASM

NASM es un ensamblador para arquitectura x86 que permite un control total del hardware. NASM ofrece:

- Acceso directo a registros (EAX, EBX, ECX...)
- Manipulación de memoria
- Control del flujo del programa
- Optimización a bajo nivel

2.3 Convención de llamadas (cdecl)

En C bajo Windows de 32 bits, se utiliza la convención **cdecl**, donde:

- Los parámetros se pasan por la pila
- El valor de retorno se almacena en EAX
- El llamador limpia la pila después de la llamada
- La dirección de la pila crece hacia valores menores

Las funciones en ensamblador deben declararse con un guion bajo: `_sumar`, `_restar`, `_multiplicar`, etc.

2.4 La Pila (Stack)

La pila permite almacenar variables temporales, direcciones de retorno y parámetros. Lo controlamos mediante los registros:

- **ESP** → puntero de pila
- **EBP** → base del marco de activación

Todas las funciones NASM del proyecto siguen el estándar:

```
push ebp
```

```
mov ebp, esp
```

```
mov esp, ebp
```

```
pop ebp
```

```
ret
```

2.5 Macros en NASM

Se utilizan para simplificar bloques repetitivos. En este proyecto se emplean dos macros:

`SAVE` ; prepara la pila

`RESTORE` ; restaura la pila

Esto hace que nuestro código sea limpio y modular.

3. Diseño del Proyecto

El diseño del sistema se estructuró en dos módulos principales, uno escrito en C y otro implementado en NASM. El módulo en C actúa como la interfaz del usuario al presentar un menú con diversas opciones, recibir datos numéricos o cadenas de texto y enviar esos valores a las rutinas apropiadas escritas en ensamblador. Cada selección del menú desencadena una llamada directa hacia una función externa definida en NASM, lo que permite mantener una separación clara entre la lógica de interacción y la lógica operativa. Por su parte, el módulo NASM contiene tanto funciones aritméticas como funciones orientadas al manejo de cadenas, todas cuidadosamente implementadas para cumplir con los requisitos académicos como el uso de registros específicos, ciclos de control y manipulación explícita de la pila. Esta división de responsabilidades facilita la claridad del proyecto y demuestra la cooperación entre alto nivel y bajo nivel dentro de un mismo ejecutable.

3.1 Módulo en C (main.c)

Responsabilidades:

- Mostrar menú principal
- Capturar entradas del usuario
- Validar opciones
- Llamar a funciones implementadas en NASM
- Mostrar resultados en pantalla

Menú requerido:

1. Sumar dos números
2. Restar

3. Multiplicar
4. Dividir
5. Contar caracteres de una cadena
6. Convertir una cadena a MAYÚSCULAS
7. Salir

Todas las llamadas a NASM utilizan declaraciones extern.

3.2 Módulo NASM (operaciones.asm)

Incluye 6 rutinas:

Sumar:

Usa EAX + EBX

Restar:

Usa SUB

Multiplicar:

Usa IMUL

Dividir:

Se usa CDQ + IDIV

Incluye manejo de divisor cero.

Contar caracteres:

Simulamos strlen con comparación a 0x00.

Convertir a MAYÚSCULAS:

Convierte caracteres del rango a–z restando 32.

3.3 Macros

Para estandarización del manejo de pila se definen:

```
%macro SAVE 0
```

```
    push ebp
```

```
    mov ebp, esp
```

```
%endmacro
```

```
%macro RESTORE 0
```

```
    mov esp, ebp
```

```
    pop ebp
```

```
%endmacro
```

4. Código fuente:

Main.c

```
#include <stdio.h>
#include <string.h>

// Declaracion de las funciones que tenemos en el modulo de NASM.
extern int sumar(int a, int b);
extern int restar(int a, int b);
extern int multiplicar(int a, int b);
extern int dividir(int a, int b);
extern int contarCaracteres(char *cadena);
extern void convertirMayusculas(char *cadena);

int main() {
    int opcion;
    int a, b, resultado;
    char texto[200];

    do {
        // Menu principal
        printf("\n MENU MINI INTERPRETE C + NASM \n");
        printf("1. Sumar\n");
        printf("2. Restar\n");
        printf("3. Multiplicar\n");
        printf("4. Dividir\n");
        printf("5. Contar caracteres\n");
        printf("6. Convertir a MAYUSCULAS\n");
        printf("7. Salir\n");
        printf("Selecciona opcion: ");
        scanf("%d", &opcion);

        switch(opcion) {

            case 1:
                // Captura de numeros y llama a la función NASM
                printf("Ingresa A: "); scanf("%d", &a);
                printf("Ingresa B: "); scanf("%d", &b);
                resultado = sumar(a, b);
                printf("Resultado = %d\n", resultado);
                break;

            case 2:
                // Llamada a la funcion restar en NASM
                printf("Ingresa A: "); scanf("%d", &a);
                printf("Ingresa B: "); scanf("%d", &b);
                resultado = restar(a, b);
                printf("Resultado = %d\n", resultado);
                break;
```

```
case 3:
    // Llamada a multiplicar en NASM
    printf("Ingresa A: "); scanf("%d", &a);
    printf("Ingresa B: "); scanf("%d", &b);
    resultado = multiplicar(a, b);
    printf("Resultado = %d\n", resultado);
    break;
```

```
case 4:
    // Llamada a dividir, con manejo de cero dentro del NASM
    printf("Ingresa A: "); scanf("%d", &a);
    printf("Ingresa B: "); scanf("%d", &b);
    resultado = dividir(a, b);
    printf("Resultado = %d\n", resultado);
    break;
```

```
case 5:
    // Envio de una cadena para contar sus caracteres
    printf("Ingresa texto: ");
    scanf(" %[^\n]", texto);
    resultado = contarCaracteres(texto);
    printf("Total caracteres = %d\n", resultado);
    break;
```

```
case 6:
    // Conversion de una cadena a mayúsculas mediante NASM
    printf("Ingresa texto: ");
    scanf(" %[^\n]", texto);
    convertirMayusculas(texto);
    printf("MAYUSCULAS = %s\n", texto);
    break;
```

```
case 7:
    // Salida del programa
    printf("Saliendo...\n");
    break;
```

```
default:
    // Opcion no valida
    printf("Opcion invalida.\n");
```

```
}
```

```
} while(opcion != 7);
```

```
return 0;
}
```

operaciones.asm

```
%include "macros.inc"
```

```
global _sumar, _restar, _multiplicar, _dividir  
global _contarCaracteres, _convertirMayusculas
```

```
SECTION .text
```

```
; -----  
; SUMAR  
; Recibe dos numeros desde C y devuelve su suma.  
; Los valores llegan en [ebp+8] y [ebp+12].  
; -----  
_sumar:  
    SAVE  
    mov eax, [ebp+8]    ; primer numero  
    mov ebx, [ebp+12]   ; segundo numero  
    add eax, ebx        ; suma en eax  
    RESTORE  
    ret  
  
; -----  
; RESTAR  
; Resta b a a. Igual que arriba, se toman desde la pila.  
; -----  
_restar:  
    SAVE  
    mov eax, [ebp+8]  
    mov ebx, [ebp+12]  
    sub eax, ebx  
    RESTORE  
    ret  
  
; -----  
; MULTIPLICAR  
; Multiplica los dos valores recibidos.  
; El resultado queda en eax automaticamente.  
; -----  
_multiplicar:  
    SAVE  
    mov eax, [ebp+8]  
    mov ebx, [ebp+12]  
    imul ebx  
    RESTORE  
    ret  
  
; -----  
; DIVIDIR  
; Division con verificacion para evitar dividir entre cero.  
; Si el divisor es cero, se devuelve 0.  
; -----  
_dividir:  
    SAVE  
    mov eax, [ebp+8]    ; dividendo
```

```

    mov ebx, [ebp+12]    ; divisor

    cmp ebx, 0           ; si el divisor es cero
    je .err              ; si sí, regresar 0

    cdq                  ; extender signo para la division
    idiv ebx              ; dividir eax / ebx
    jmp .fin

.err:
    mov eax, 0           ; valor por defecto

.fin:
    RESTORE
    ret

; -----
; CONTAR CARACTERES
; Recorre la cadena hasta encontrar el 0 final.
; Va sumando en ecx el total de caracteres.
; -----
_contarCaracteres:
    SAVE
    mov esi, [ebp+8]     ; puntero a la cadena
    xor ecx, ecx         ; contador en 0

.loop:
    mov al, [esi]        ; obtener el caracter actual
    cmp al, 0            ; fin de cadena
    je .fin

    inc ecx              ; sumar al conteo
    inc esi              ; avanzar
    jmp .loop

.fin:
    mov eax, ecx         ; devolver el total
    RESTORE
    ret

; -----
; CONVERTIR A MAYUSCULAS
; Recorre la cadena y convierte cada letra de 'a' a 'z'
; utilizando la diferencia de 32 en ASCII.
; -----
_convertirMayusculas:
    SAVE
    mov esi, [ebp+8]

.loop2:
    mov al, [esi]        ; obtener caracter
    cmp al, 0            ; fin de cadena
    je .fin2

    cmp al, 'a'          ; menor a 'a' → no tocar
    jl .next
    cmp al, 'z'          ; mayor a 'z' → no tocar

```



```

    jg .next

    sub al, 32      ; convertir a mayúscula
    mov [esi], al

.next:
    inc esi
    jmp .loop2

.fin2:
    RESTORE
    ret

```

macros.inc

```

; -----
; Macro SAVE
; Este lo usaremos al inicio de cada funcion.
; Este nos va a servir para crear un nuevo marco de pila para
; poder acceder a los parametros con [ebp+8], etc.
; -----
%macro SAVE 0
    push ebp      ; guardar el valor anterior de ebp
    mov ebp, esp  ; establecer este ebp como base de la funcion
%endmacro

; -----
; Macro RESTORE
; Este lo vamos a usar al final de la funcion para dejar la pila
; como estaba antes de entrar.
; -----
%macro RESTORE 0
    mov esp, ebp  ; recuperar el puntero de pila original
    pop ebp       ; restaurar el ebp anterior
%endmacro

```

5. Pruebas

se realizaron ejecutando el programa final en Windows, seleccionando cada una de las opciones del menú y verificando que las operaciones aritméticas devolvieran resultados correctos bajo distintos valores, incluyendo casos con números negativos y divisiones con diferentes combinaciones. En el caso de las funciones de cadenas se probaron textos de distintas longitudes para asegurar que el conteo detectara correctamente la terminación y que la conversión a mayúsculas afectara únicamente a caracteres válidos. Se realizaron capturas de pantalla mostrando la ejecución real del programa, lo que evidencia el funcionamiento adecuado de cada módulo y confirma la correcta comunicación entre C y NASM durante toda la ejecución.

Capturas:

Nombre	Fecha de modificación	Tipo	Tamaño
Al principio de esta semana			
main.c	13/11/2025 07:14 p. m.	Archivo C	3 KB
interprete	13/11/2025 07:06 p. m.	Aplicación	425 KB
main.obj	13/11/2025 07:05 p. m.	Archivo OBJ	3 KB
operaciones.obj	13/11/2025 07:05 p. m.	Archivo OBJ	1 KB
operaciones.asm	13/11/2025 07:05 p. m.	Archivo ASM	2 KB
macros.inc	13/11/2025 07:05 p. m.	Archivo INC	1 KB

Menu y opcion suma:

```
MENU MINI INTERPRETE C + NASM
1. Sumar
2. Restar
3. Multiplicar
4. Dividir
5. Contar caracteres
6. Convertir a MAYUSCULAS
7. Salir
Selecciona opcion: 1
Ingresa A: 30
Ingresa B: 20
Resultado = 50
```

```
MENU MINI INTERPRETE C + NASM
1. Sumar
2. Restar
3. Multiplicar
4. Dividir
5. Contar caracteres
6. Convertir a MAYUSCULAS
7. Salir
Selecciona opcion: 1
Ingresa A: -30
Ingresa B: 20
Resultado = -10
```

Opcion resta:

```
MENU MINI INTERPRETE C + NASM
1. Sumar
2. Restar
3. Multiplicar
4. Dividir
5. Contar caracteres
6. Convertir a MAYUSCULAS
7. Salir
Selecciona opcion: 2
Ingresa A: 300
Ingresa B: 250
Resultado = 50
```

Opcion multiplicar:

```
MENU MINI INTERPRETE C + NASM
1. Sumar
2. Restar
3. Multiplicar
4. Dividir
5. Contar caracteres
6. Convertir a MAYUSCULAS
7. Salir
Selecciona opcion: 3
Ingresa A: 15
Ingresa B: 8
Resultado = 120
```

Opcion dividir:

```
MENU MINI INTERPRETE C + NASM
1. Sumar
2. Restar
3. Multiplicar
4. Dividir
5. Contar caracteres
6. Convertir a MAYUSCULAS
7. Salir
Selecciona opcion: 4
Ingresa A: 60
Ingresa B: 5
Resultado = 12
```

Opcion contar caracteres:

```
MENU MINI INTERPRETE C + NASM
1. Sumar
2. Restar
3. Multiplicar
4. Dividir
5. Contar caracteres
6. Convertir a MAYUSCULAS
7. Salir
Selecciona opcion: 5
Ingresa texto: Traductores de bajo nivel
Total caracteres = 25
```

Opcion convertir a mayusculas:

```
MENU MINI INTERPRETE C + NASM
1. Sumar
2. Restar
3. Multiplicar
4. Dividir
5. Contar caracteres
6. Convertir a MAYUSCULAS
7. Salir
Selecciona opcion: 6
Ingresa texto: proyecto final
MAYUSCULAS = PROYECTO FINAL
```

Comandos de compilacion en CMD:

```
nasm -f win32 operaciones.asm -o operaciones.obj
```

```
gcc -m32 -c main.c -o main.obj
```

```
gcc -m32 main.obj operaciones.obj -o interprete.exe
```

6. Conclusiones

En este proyecto integramos un módulo en lenguaje C con un conjunto de rutinas escritas en ensamblador NASM, y con esto logramos una comunicación entre ambos y demostrando el funcionamiento interno de las convenciones de llamada en arquitectura x86. En este proyecto se implementaron operaciones basicas matematicas como suma multiplicacion division y resta, y de manejo de cadenas que funcionan de buena manera gracias al uso explícito de registros, ciclos de control, manipulación de direcciones de memoria y administración correcta de la pila. Las pruebas realizadas evidencian que cada función responde correctamente a las entradas proporcionadas en nuestro teclado, confirmando la correcta transferencia de parámetros y la coherencia en los resultados que nos da el programa. Y ya por ultimo, el proyecto nos demuestra que si podemos combinar el lenguaje de C y NASM, Haciendo que tengamos un mejor control y limpieza de codigo, ademas que también nos ayuda a generar soluciones eficientes que aprovechan tanto la flexibilidad del alto nivel como el control detallado del bajo nivel.

Anexos:

TDM-GCC: <https://jmeubank.github.io/tdm-gcc/download/>

Nasm: <https://www.nasm.us/pub/nasm/releasebuilds/2.16.03/win64/>

Drive con el proyecto: https://drive.google.com/drive/folders/1u6sSSNXplkoHDZh9749Nt-dhahMFjhQZ?usp=drive_link